

**UNIVERSIDAD AUTÓNOMA GABRIEL RENÉ MORENO**  
**FACULTAD DE INGENIERÍA EN CIENCIAS DE LA COMPUTACIÓN Y**  
**TELECOMUNICACIONES**



**GeneradorUML**

Herramienta CASE colaborativa para modelado UML y generación de software

Documentación del proyecto — Ingeniería de Software 1

Integrante: Caballero Pereira Alejandro

Docente: Ing. Rolando Martinez

Semestre 2/2026

Santa Cruz – Bolivia

22 de septiembre de 2026

## Índice

|                                                                    |    |
|--------------------------------------------------------------------|----|
| Índice.....                                                        | 2  |
| Lista de figuras.....                                              | 6  |
| Resumen.....                                                       | 7  |
| 1 Introducción y definición del proyecto .....                     | 8  |
| 1.1 Problema y contexto .....                                      | 8  |
| 1.2 Objetivo general.....                                          | 8  |
| 1.3 Objetivos específicos .....                                    | 8  |
| 1.4 Alcance y límites.....                                         | 9  |
| 2 Fundamentación teórica aplicada.....                             | 9  |
| 2.1 Ingeniería de software asistida por computadora.....           | 9  |
| 2.2 Desarrollo basado en componentes y reutilización.....          | 10 |
| 2.3 Arquitectura y capacidad de cambio.....                        | 10 |
| 2.4 UML como lenguaje de modelado.....                             | 11 |
| 2.5 Proceso Unificado iterativo e incremental .....                | 11 |
| 2.6 Reconocimiento de texto y transcripción local .....            | 12 |
| 2.7 Interfaz móvil reutilizada .....                               | 12 |
| 2.8 Inteligencia artificial en el desarrollo y en el producto..... | 12 |
| 2.9 Calidad y productividad mediante reutilización .....           | 13 |
| 2.10 Persistencia de las aplicaciones generadas .....              | 14 |

|                                                        |    |
|--------------------------------------------------------|----|
| 3 Requisitos y análisis.....                           | 14 |
| 3.1 Requisitos funcionales .....                       | 14 |
| 3.2 Requisitos de calidad .....                        | 15 |
| 3.3 Prioridad y criterio de aceptación .....           | 15 |
| 3.4 Actores .....                                      | 15 |
| 3.5 Historias de usuario.....                          | 16 |
| 3.6 Casos de uso.....                                  | 18 |
| 4 Arquitectura y componentes reutilizados.....         | 23 |
| 4.1 Distribución de responsabilidades .....            | 23 |
| 4.2 Inventario de reutilización .....                  | 23 |
| 4.3 Decisiones y consecuencias .....                   | 24 |
| 5 Diagrama de clases y diseño estructural .....        | 25 |
| 5.1 Lectura de la notación.....                        | 25 |
| 5.2 Núcleo del modelo implementado .....               | 25 |
| 5.3 Relaciones y extremos .....                        | 26 |
| 5.4 Servicios que consumen el modelo.....              | 28 |
| 5.5 Ejemplo de negocio y reglas .....                  | 29 |
| 5.6 Reglas y trazabilidad estructural .....            | 30 |
| 5.7 Transformación del modelo a JPA y PostgreSQL ..... | 31 |
| 5.8 Criterio de revisión de UML 2.5.1.....             | 32 |

|     |                                                |    |
|-----|------------------------------------------------|----|
| 6   | Diseño dinámico y despliegue .....             | 33 |
| 7   | Registro del Proceso Unificado .....           | 37 |
| 7.1 | Criterio del registro .....                    | 37 |
| 7.2 | Iteraciones reconstruidas.....                 | 37 |
| 7.3 | Artefactos por disciplina .....                | 39 |
| 7.4 | Registro de correcciones como incrementos..... | 39 |
| 7.5 | Ciclo de vida y evidencia del proceso.....     | 39 |
| 8   | Manual de usuario y tutorial .....             | 40 |
| 8.1 | Antes de comenzar .....                        | 40 |
| 8.2 | Tutorial de creación móvil .....               | 41 |
| 8.3 | Tutorial web .....                             | 41 |
| 8.4 | Tutorial de voz .....                          | 42 |
| 8.5 | Tutorial de fotografía y corrección .....      | 42 |
| 8.6 | Tutorial de colaboración y reconexión .....    | 42 |
| 8.7 | Exportar y generar.....                        | 43 |
| 8.8 | Resolución de problemas .....                  | 43 |
| 8.9 | Aprendizaje asistido y límites actuales .....  | 44 |
| 9   | Verificación y aceptación del producto .....   | 45 |
| 9.1 | Protocolo final de aceptación.....             | 45 |
| 9.2 | Comprobación de backend generado y API.....    | 46 |

|                                                             |    |
|-------------------------------------------------------------|----|
| 9.3 Criterio de cierre .....                                | 46 |
| 10 Conclusiones y continuidad .....                         | 46 |
| Referencias.....                                            | 48 |
| Anexo A Inventario de evidencias .....                      | 49 |
| Anexo B Guion de demostración.....                          | 50 |
| Anexo C Evidencias de prueba y registro de aceptación ..... | 51 |
| E01 — Importación de clases y atributos desde imagen .....  | 51 |
| E02 — Métodos y visibilidad de la imagen.....               | 51 |
| E03 — Edición manual y persistencia móvil.....              | 51 |
| E04 — Voz con interpretación y corrección.....              | 52 |
| E05 — Regresión automatizada.....                           | 52 |
| E06 — Límites de la evidencia .....                         | 52 |
| Anexo D Glosario .....                                      | 53 |
| Anexo E Preparación de la defensa y trazabilidad.....       | 54 |

### **Lista de figuras**

Figura 1. Casos de uso y actores

Figura 2. Componentes y responsabilidades

Figura 3. Modelo de clases del núcleo

Figura 4. Relaciones y extremos del modelo

Figura 5. Servicios y dependencias del modelo

Figura 6. Ejemplo didáctico de clases de negocio

Figura 7. Despliegue lógico web y Android

Figura 8. Secuencia de importación de fotografía

Figura 9. Secuencia de sincronización y conflictos

Figura 10. Actividad de generación y validación

Figura 11. Estados conceptuales de sincronización

## Resumen

GeneradorUML es una herramienta de ingeniería de software asistida por computadora orientada a construir diagramas de clases, revisarlos y transformarlos en proyectos de software. Integra un editor web colaborativo y un complemento Android del mismo editor. Permite introducir información mediante controles visuales, formularios, voz e imágenes. Los proyectos Spring Boot y Flutter generados constituyen una salida del sistema, distinta de la APK Graficador UML. Este informe describe el problema, la fundamentación utilizada, los requisitos, el diseño mediante diagramas UML, la reconstrucción del proceso iterativo e incremental y los procedimientos de uso. El contenido se apoya en documentación técnica consultada, código del repositorio y pruebas registradas. Se identifican expresamente los límites de reconocimiento de voz e imágenes y las verificaciones pendientes de dispositivos y herramientas externas. La documentación está completa como versión de revisión académica; no acredita una aceptación del producto que todavía no se ha realizado.

Palabras clave: CASE, UML, Proceso Unificado, componentes, colaboración, generación de código.

## **1 Introducción y definición del proyecto**

### **1.1 Problema y contexto**

La elaboración de diagramas puede comenzar en una pizarra, continuar en un teléfono y refinarse en una computadora. Transcribir manualmente esos elementos, mantener versiones diferentes y trasladar la estructura al código introduce trabajo repetido y oportunidades de error. El proyecto busca mantener un modelo común entre esas actividades, con mecanismos de revisión que permitan corregir interpretaciones incorrectas.

El contexto académico exige una herramienta demostrable y una explicación verificable de su construcción. Por ello, el informe diferencia requisitos, decisiones de diseño, funciones implementadas y resultados de pruebas. No se considera que la existencia de un botón o de una clase en el código sea suficiente para demostrar el cumplimiento de un requisito.

### **1.2 Objetivo general**

Desarrollar y documentar una herramienta CASE para crear y editar diagramas de clases desde web y Android, colaborar sobre un modelo compartido e intercambiar y transformar ese modelo en código fuente.

### **1.3 Objetivos específicos**

- Mantener una representación de clases, atributos, operaciones y relaciones utilizable por el editor y los generadores.
- Facilitar la entrada móvil mediante formularios, transcripción de voz y lectura de imágenes con revisión.

- Conservar cambios locales y reconciliarlos con el proyecto compartido al recuperar la conexión.
- Ofrecer adaptadores de archivos y generación de proyectos con controles de coherencia.
- Registrar requisitos, decisiones, pruebas y limitaciones mediante un proceso iterativo e incremental.

## **1.4 Alcance y límites**

El alcance funcional se concentra en diagramas de clases. Los demás diagramas incluidos en este informe explican el diseño del software; no significan que el editor permita crear todos esos tipos de diagramas. Tampoco se afirma implementación de toda la especificación UML 2.5.1 ni certificación formal de conformidad.

La transcripción local y el intérprete de comandos no equivalen a una conversación general sin conexión. Gemini es un servicio en línea. La generación de firmas de métodos no permite deducir automáticamente toda la lógica de negocio. La interoperabilidad debe verificarse también dentro de las herramientas externas, además de las pruebas internas de archivos.

## **2 Fundamentación teórica aplicada**

### **2.1 Ingeniería de software asistida por computadora**

Las herramientas CASE apoyan y automatizan parcialmente actividades de la ingeniería de software, como especificación, diseño, implementación y pruebas (IEEE, s. f.). La palabra parcialmente es relevante: la herramienta facilita tareas, pero no sustituye la decisión del modelador sobre el significado del dominio.

En GeneradorUML, la aplicación de este enfoque se observa en el editor, el validador, los adaptadores y los generadores. El modelo estructurado es el elemento que permite pasar de la

representación gráfica a archivos de intercambio y fuentes. Esta clasificación describe el propósito del proyecto; no lo equipara a una plataforma industrial certificada ni implica que cubra todo el ciclo de vida.

## **2.2 Desarrollo basado en componentes y reutilización**

Clements (1995) presenta el desarrollo basado en componentes como evolución de la reutilización de software. Para este proyecto, reutilizar significa integrar capacidades existentes mediante contratos definidos y conservar código propio para la coordinación y las reglas específicas del modelado.

El servidor utiliza FastAPI y bibliotecas de tratamiento de datos; Android utiliza WebView; el lector utiliza Tesseract y la transcripción utiliza un modelo ejecutado con Transformers.js. Estos elementos no se desarrollaron desde cero. Su integración exige revisar entradas, salidas, versiones, disponibilidad y errores. La presencia de bibliotecas no demuestra por sí sola una arquitectura desacoplada: esa calidad debe analizarse en las dependencias concretas.

## **2.3 Arquitectura y capacidad de cambio**

La arquitectura expresa decisiones sobre estructura y comportamiento que permiten razonar sobre cualidades del sistema, entre ellas modificabilidad, disponibilidad y seguridad (Software Engineering Institute, s. f.). En este proyecto, separar modelo, validación, adaptadores y generadores busca reducir el impacto de añadir un formato o corregir una transformación.

La reutilización del lector entre web y Android evita implementar reglas diferentes para la misma fotografía. Sin embargo, existen dependencias entre módulos JavaScript y estado compartido que limitan el aislamiento. La escalabilidad es un objetivo de evolución, no un

resultado medido: no se dispone de evidencia suficiente para afirmar un número de usuarios simultáneos ni rendimiento constante.

## **2.4 UML como lenguaje de modelado**

La especificación de referencia es UML 2.5.1, publicada por Object Management Group en 2017. Se utiliza para describir estructura e interacción mediante vistas complementarias (Object Management Group [OMG], 2017). El informe emplea clases, casos de uso, componentes, despliegue, secuencias, actividades y estados según el propósito de cada figura.

La estructura implementada por GeneradorUML representa un subconjunto de conceptos del lenguaje. Una propiedad de configuración que indique la versión UML no prueba conformidad. Los diagramas de este informe son elaboraciones del proyecto; PlantUML se utiliza como herramienta de representación y no como certificado de corrección semántica.

## **2.5 Proceso Unificado iterativo e incremental**

El Proceso Unificado organiza el desarrollo mediante las fases Inicio, Elaboración, Construcción y Transición, divididas en iteraciones (IBM, s. f.). En este informe, los casos de uso ayudan a expresar objetivos de los actores y las iteraciones agrupan incrementos verificables. Las historias de usuario se incluyen como apoyo para priorizar y formular criterios de aceptación.

El registro disponible permite reconstruir entregas y correcciones, pero no demostrar que todas las actividades metodológicas se realizaron en una fecha específica. Por ello, no se inventa un cronograma histórico. La fase de Transición permanece abierta mientras falten aceptación y empaquetado de los cambios recientes.

## **2.6 Reconocimiento de texto y transcripción local**

Tesseract es un motor de reconocimiento óptico que extrae texto de imágenes y requiere recursos de idioma (Tesseract OCR, s. f.). En el editor, extraer caracteres es solo una parte del problema: después deben distinguirse clases, atributos, métodos y conexiones. Una lectura textual correcta no acredita interpretación completa de las relaciones UML.

Transformers.js permite configurar rutas de modelos locales y deshabilitar su carga remota (Hugging Face, s. f.). El trabajador de voz del proyecto utiliza estas opciones y ejecuta la transcripción mediante WASM. El texto obtenido pasa a otra etapa de interpretación. Separar ambas etapas permite identificar si el fallo proviene del audio reconocido o de la orden interpretada.

## **2.7 Interfaz móvil reutilizada**

WebView permite incorporar contenido web dentro de una aplicación Android y vincularlo con capacidades de la aplicación anfitriona (Android Developers, s. f.). GeneradorUML utiliza este enfoque para distribuir el editor en una APK y reutilizar su lógica web. No corresponde describir esta APK como una aplicación Flutter nativa.

La decisión reduce duplicación, pero exige comprobar permisos, acceso a archivos, micrófono, cámara y comportamiento sin conexión en el dispositivo. Que una función opere en el navegador de escritorio no demuestra automáticamente su funcionamiento en WebView.

## **2.8 Inteligencia artificial en el desarrollo y en el producto**

En el desarrollo se utilizaron asistentes de código de Codex y Antigravity para proponer cambios, diagnosticar fallos, preparar pruebas y apoyar la documentación. Sus respuestas se trataron como propuestas que requieren contraste con el repositorio y los resultados de ejecución.

Esta asistencia no sustituye la responsabilidad del autor de explicar el código ni acredita por sí sola calidad o autoría.

En el producto, la transcripción convierte audio en texto; el OCR recupera texto de imágenes, y la interpretación transforma solicitudes en acciones sobre el modelo. El motor local y los comandos admitidos permiten un alcance offline limitado. Gemini corresponde al servicio conversacional en línea: no se presenta como IA local ni se presupone equivalencia entre ambas capacidades.

Se reutilizaron modelos preentrenados. No se documenta entrenamiento propio porque no existe evidencia de un conjunto de entrenamiento, procedimiento, parámetros y evaluación reproducible. Las correcciones del intérprete y el preprocesamiento de imágenes son ajustes del sistema, no entrenamiento de un modelo.

## **2.9 Calidad y productividad mediante reutilización**

Las librerías aportan funciones invocadas por el programa; por ejemplo, Jinja2 interviene en la generación y las herramientas de procesamiento de imágenes apoyan el reconocimiento. Los componentes agrupan capacidades con una interfaz, como el lector local, el adaptador XMI y los generadores. Los frameworks organizan una parte de la aplicación: FastAPI estructura el backend del editor y Spring Boot el backend producido. Estas categorías describen distintos niveles de reutilización, no una clasificación excluyente de todos los paquetes.

La productividad se evidencia en la transformación repetible de un modelo en archivos y en el uso del mismo lector para web y Android. La calidad se apoya en validaciones, contratos, pruebas de regresión y revisión del resultado. No se dispone de una comparación controlada de horas o costos; por ello no se atribuyen porcentajes de mejora ni ausencia total de fallos.

## **2.10 Persistencia de las aplicaciones generadas**

Spring Data JPA proporciona abstracciones de repositorio para el acceso a entidades persistentes y reduce código repetitivo de acceso a datos (Spring, s. f.). En GeneradorUML, esta capacidad pertenece a las aplicaciones Spring Boot generadas. No debe confundirse con SQLite, utilizado por el servicio colaborativo del editor. La configuración generada incluye PostgreSQL como destino de persistencia.

## **3 Requisitos y análisis**

La fuente primaria de requisitos es la descripción proporcionada por el usuario durante el desarrollo y la revisión documental del 21 de septiembre de 2026. No se han inventado encuestas ni entrevistas. Los requisitos documentales incluyen fundamentación aplicada, componentes reutilizados, arquitectura, capítulo de clases UML 2.5 o superior, registro PUD, manual, tutorial, anexos y presentación APA.

### **3.1 Requisitos funcionales**

- RF01 Gestionar clases, atributos y operaciones desde las interfaces del editor.
- RF02 Definir relaciones con extremos, tipo y multiplicidad cuando corresponda.
- RF03 Compartir proyectos con roles y colaboración simultánea.
- RF04 Capturar voz, interpretar órdenes y permitir corregir resultados.
- RF05 Leer imágenes y revisar el modelo antes de importarlo.
- RF06 Persistir cambios sin conexión y resolver diferencias al reconectar.
- RF07 Importar y exportar modelos para intercambio.
- RF08 Generar fuentes Spring Boot, Flutter y artefactos asociados.
- RF09 Consultar historial y restaurar revisiones con autorización.

### 3.2 Requisitos de calidad

- RQ01 Comprensibilidad: mostrar estados de escucha, procesamiento, guardado y error con acciones claras.

- RQ02 Integridad: no reemplazar silenciosamente el diagrama ni omitir elementos inválidos sin advertencia.

- RQ03 Autorización: verificar permisos en el servidor; deshabilitar un botón no basta.

- RQ04 Modificabilidad: mantener responsabilidades reconocibles y pruebas de regresión para cambios.

- RQ05 Funcionamiento local: conservar recursos y borradores necesarios; declarar qué funciones requieren red.

- RQ06 Interoperabilidad: verificar estructura y significado, no solo que el archivo se pueda abrir.

### 3.3 Prioridad y criterio de aceptación

Modelado, colaboración, voz, foto, funcionamiento local e intercambio son requisitos centrales, no extras que sustituyan unos a otros. La aceptación requiere un recorrido completo con resultados observables. Los casos negativos son parte de la aceptación: una imagen ilegible, un enlace sin permiso o un conflicto no deben producir un mensaje falso de éxito.

### 3.4 Actores

El límite comprende el editor web, el complemento Android y el backend GeneradorUML. El backend es un componente interno, no un actor externo.

La tabla 1 presenta actores del sistema.

**Tabla 1**

*Actores del sistema*

| Actor                      | Responsabilidad                                          |
|----------------------------|----------------------------------------------------------|
| Modelador                  | Crear, revisar, importar y generar desde diagramas       |
| Administrador del proyecto | Compartir acceso y gestionar restauraciones autorizadas  |
| Colaborador editor         | Modificar un proyecto compartido                         |
| Visualizador               | Consultar el proyecto sin modificarlo                    |
| Servicio Gemini            | Interpretación conversacional en línea cuando se utiliza |

Una persona puede actuar como modelador desde web o Android. StarUML y Enterprise Architect intervienen mediante archivos intercambiados; no se presupone una conexión directa con sus procesos.

### 3.5 Historias de usuario

Las historias complementan los casos de uso para ordenar incrementos; no sustituyen su especificación. Los criterios son objetivos de aceptación, no afirmaciones de que todos estén aprobados.

La tabla 2 presenta historias de usuario y criterios de aceptación.

**Tabla 2**

*Historias de usuario y criterios de aceptación*

| ID    | Historia de usuario                                               | Criterio de aceptación                                                | Caso  |
|-------|-------------------------------------------------------------------|-----------------------------------------------------------------------|-------|
| HU-01 | Como modelador quiero editar clases para representar mi dominio   | Crear clase con atributos y métodos; conservarla al guardar y reabrir | CU-01 |
| HU-02 | Como modelador quiero conectar clases para expresar su estructura | Elegir extremos, tipo y multiplicidad; rechazar herencia cíclica      | CU-02 |

| ID    | Historia de usuario                                                   | Criterio de aceptación                                                     | Caso  |
|-------|-----------------------------------------------------------------------|----------------------------------------------------------------------------|-------|
| HU-03 | Como administrador quiero compartir un enlace para trabajar con otros | Editor puede modificar; visualizador no puede hacerlo en el servidor       | CU-03 |
| HU-04 | Como colaborador quiero conservar cambios simultáneos                 | Combinar cambios independientes y presentar conflictos del mismo dato      | CU-04 |
| HU-05 | Como usuario móvil quiero dictar para reducir escritura               | Transcribir, interpretar y confirmar lo que realmente se aplica            | CU-05 |
| HU-06 | Como modelador quiero cargar una foto para recuperar un diagrama      | Revisar clases, atributos, métodos y conexiones antes de importar          | CU-06 |
| HU-07 | Como usuario móvil quiero corregir una lectura con voz                | Cambiar el atributo de la clase indicada sin alterar otras clases          | CU-06 |
| HU-08 | Como usuario móvil quiero trabajar sin red                            | Guardar, cerrar, reabrir y reconciliar al recuperar conexión               | CU-04 |
| HU-09 | Como modelador quiero intercambiar archivos con otras herramientas    | Comparar clases y relaciones después de exportar e importar                | CU-07 |
| HU-10 | Como desarrollador quiero generar una base de aplicación              | Obtener fuentes Spring Boot y Flutter con estructura y rutas coherentes    | CU-08 |
| HU-11 | Como administrador quiero restaurar una revisión                      | Autorizar restauración, registrar nueva revisión y notificar colaboradores | CU-09 |

Requisitos transversales: procesamiento local para el flujo offline; mensajes de estado comprensibles; conservación del borrador ante errores; permisos comprobados en servidor; revisión de interpretaciones ambiguas. No se fijan cifras de latencia o precisión sin mediciones.

### 3.6 Casos de uso

#### ***CU-01. Gestionar elementos del diagrama***

Actor: modelador con permiso de edición. Precondición: proyecto abierto. Disparador: crear o editar un elemento.

1. El actor introduce nombre, atributos y operaciones mediante los controles disponibles.
2. El sistema valida los datos y actualiza la representación.
3. El sistema guarda y notifica cambios según conectividad.

Alternativas: dato inválido: mostrar error sin aplicar; solo lectura: rechazar modificación. Postcondición de éxito: modelo actualizado. Error de almacenamiento: informar que no se pudo guardar, sin afirmar persistencia.

#### ***CU-02. Definir relaciones***

Actor: modelador. Precondición: extremos existentes y permiso de edición.

1. Seleccionar clases, tipo de relación y multiplicidades aplicables.
2. Validar extremos, duplicados y restricciones de herencia.
3. Incorporar la relación y actualizar vistas.

Alternativa: ciclo o extremos inválidos: rechazar. Postcondición: relación válida registrada; las multiplicidades no se inventan si no se especifican.

#### ***CU-03. Compartir acceso***

Actor: administrador; participantes invitados. Precondición: servidor accesible y proyecto compartido.

1. El administrador elige permiso y genera enlace.

2. El participante abre el enlace.
3. El servidor verifica el acceso y entrega el modelo con su rol.

Alternativa: enlace inválido: acceso denegado; sin servidor: no generar invitación.

Postcondición: sesión autorizada. Un enlace local no garantiza acceso desde otra red.

#### ***CU-04. Editar offline y sincronizar***

Actor: colaborador editor. Precondición: proyecto cargado y almacenamiento disponible.

1. Editar sin conexión; conservar el borrador local.
2. Reabrir y recuperar el borrador.
3. Reconectar y comparar base, versión local y remota.
4. Combinar cambios independientes y confirmar su recepción.

Alternativas: mismo dato cambiado: ofrecer resolución explícita; fallo de envío: mantener pendiente; almacenamiento fallido: advertir. Postcondición: cambios reconciliados o conflicto visible, sin declarar sincronización mientras queden cambios pendientes. La aceptación física completa está pendiente.

#### ***CU-05. Modelar mediante voz***

Actor: modelador. Precondición: permiso de micrófono y recursos de transcripción disponibles.

1. Iniciar captura y mostrar estado de escucha.
2. Detener y transcribir audio.
3. Interpretar el texto mediante el asistente disponible.
4. Aplicar acciones válidas según el flujo y comunicar el resultado real.

Alternativas: sin conexión: interpretación local limitada y propuesta revisable; frase ambigua: pedir corrección; error: conservar el modelo. Postcondición: cambio confirmado o propuesta pendiente. Gemini requiere conexión; no se atribuye su capacidad al motor offline.

#### ***CU-06. Importar una fotografía y revisar***

Actor: modelador. Precondición: imagen seleccionada y recursos OCR disponibles.

1. Leer imagen localmente, corregir inclinación cuando haya evidencia y separar recuadros.
2. Presentar texto y conexiones candidatas para revisión.
3. Corregir texto, opcionalmente con comandos de voz admitidos.
4. Validar el lote e importar mediante confirmación.

Alternativas: texto ilegible o duplicado: bloquear importación; conexiones ambiguas: revisar manualmente. Postcondición: lote incorporado; deshacer permite revertirlo. En web se agrega por defecto y reemplazar exige elección explícita. Perspectiva y pizarras variadas requieren más validación.

#### ***CU-07. Intercambiar modelos***

Actor: modelador. Precondición: archivo admitido o modelo abierto.

1. Elegir formato y archivo/destino.
2. Convertir y presentar o guardar el resultado.
3. Comparar los elementos relevantes tras la importación.

Alternativa: formato o datos no admitidos: informar error. Postcondición: archivo o modelo convertido. Las pruebas internas no certifican fidelidad completa dentro de Enterprise Architect o StarUML.

**CU-08. Generar proyectos**

Actor: modelador/desarrollador. Precondición: modelo válido y backend generador disponible.

1. Seleccionar generación y opciones.
2. Validar y transformar el modelo.
3. Entregar los archivos generados.

Alternativa: validación fallida: mostrar problemas antes de generar. Postcondición: fuentes disponibles. Compilar y ejecutar es una comprobación adicional; los métodos de negocio pueden requerir implementación manual.

**CU-09. Restaurar revisión**

Actor: administrador. Precondición: historial y servidor disponibles.

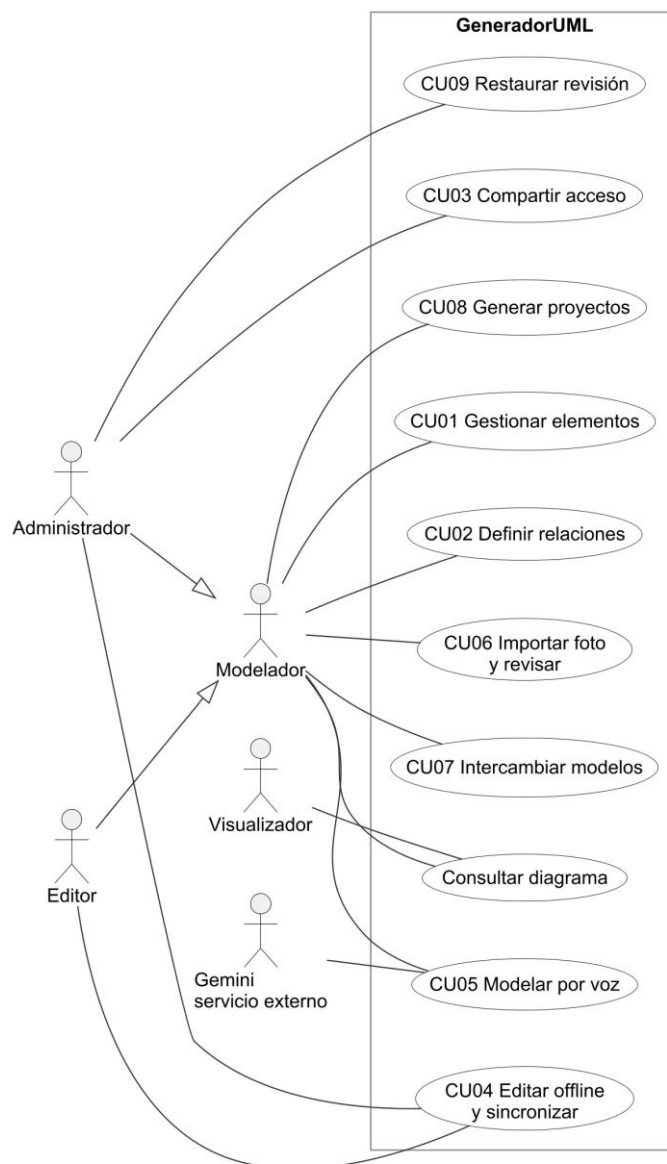
1. Consultar historial y elegir revisión.
2. Confirmar restauración.
3. Verificar autorización, crear revisión nueva y difundir el estado.

Alternativa: permiso insuficiente o revisión ausente: rechazar. Postcondición: revisión restaurada conservando trazabilidad histórica.

La figura 1 representa los actores y las funciones dentro del límite de GeneradorUML.

**Figura 1**

*Casos de uso y actores*



Nota. Elaboración propia. Representación del proyecto; los detalles omitidos no implican soporte completo de UML.

## 4 Arquitectura y componentes reutilizados

### 4.1 Distribución de responsabilidades

El frontend JavaScript presenta el lienzo SVG y los formularios. El modelo del editor se serializa para almacenamiento, intercambio y envío al servidor. FastAPI expone operaciones de colaboración y generación. SQLite almacena proyectos, tokens y revisiones del servicio colaborativo. Los modelos y borradores locales pertenecen al cliente.

La aplicación generada sigue otra arquitectura: backend Spring Boot y cliente Flutter. PostgreSQL corresponde a proyectos generados según su configuración; no debe presentarse como la base de datos colaborativa del editor, que utiliza SQLite.

### 4.2 Inventario de reutilización

La tabla 3 presenta componentes reutilizados.

**Tabla 3**

*Componentes reutilizados*

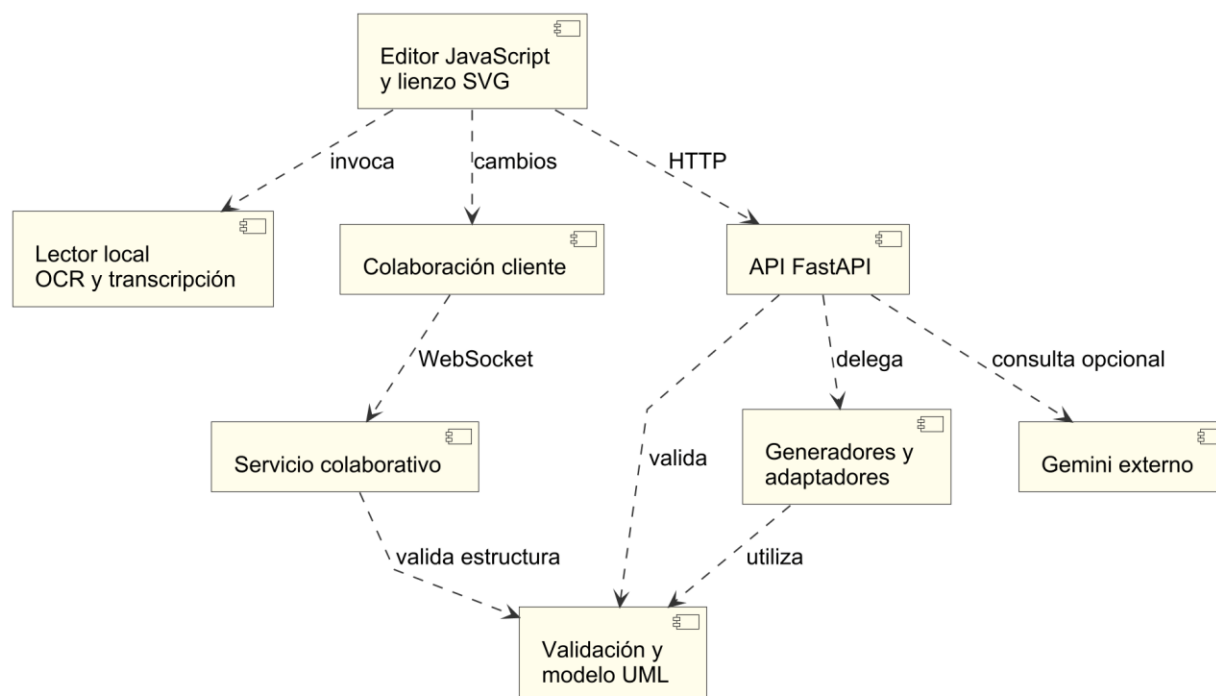
| Componente                | Uso verificado                 | Límite o responsabilidad de integración                  |
|---------------------------|--------------------------------|----------------------------------------------------------|
| FastAPI                   | API del backend                | Validar solicitudes y gestionar errores del proyecto     |
| Pydantic                  | Datos de entrada de API        | No sustituye todas las reglas semánticas UML             |
| AndroidX WebKit           | Contenedor Android del editor  | Permisos y guardado requieren prueba física              |
| Tesseract                 | OCR local y recursos asociados | Reconoce texto; estructura UML se interpreta aparte      |
| Transformers.js y Whisper | Transcripción local            | No garantiza transcripción exacta ni comprensión general |
| SQLite                    | Persistencia colaborativa      | Capacidad de carga no medida en este informe             |

| Componente                    | Uso verificado             | Límite o responsabilidad de integración |
|-------------------------------|----------------------------|-----------------------------------------|
| Adaptadores propios XMI y MDJ | Transformación de archivos | Validación externa sigue necesaria      |

La figura 2 representa la separación entre componentes locales, servidor y servicios externos.

**Figura 2**

*Componentes y responsabilidades*



Nota. Elaboración propia. Representación del proyecto; los detalles omitidos no implican soporte completo de UML.

### 4.3 Decisiones y consecuencias

La representación común del diagrama permite que un cambio de interfaz no obligue a modificar cada generador. La serialización con identificadores facilita seguir elementos entre revisiones. Los adaptadores aíslan diferencias de formato, mientras las pruebas de ida y vuelta ayudan a detectar pérdidas.

El almacenamiento local y la cola de cambios permiten continuar ante interrupciones. Su costo es una reconciliación más compleja y la necesidad de distinguir guardado local de

sincronización remota. La asistencia remota aumenta la flexibilidad de interpretación, pero agrega dependencia de conexión y de un servicio externo.

No se documenta una implantación completa de Clean Architecture, microservicios o balanceo horizontal porque el código y las pruebas revisadas no lo acreditan. La extensión futura debe conservar contratos del modelo, pruebas de compatibilidad y migración de datos.

## **5 Diagrama de clases y diseño estructural**

### **5.1 Lectura de la notación**

Cada clase se presenta con nombre y, cuando aporta información, compartimentos de atributos y operaciones. Los signos +, -, # y ~ representan visibilidad pública, privada, protegida y de paquete. Una operación indica nombre, parámetros y retorno. Los tipos Python de las figuras internas describen el código implementado; los tipos del ejemplo de negocio describen el modelo que un usuario podría crear.

Las multiplicidades expresan cuántas instancias pueden participar en un extremo. En el ejemplo de negocio, un Cliente se asocia con cero o más pedidos; cada Pedido tiene un cliente. El rombo lleno se ubica en Pedido para expresar composición con sus líneas. En el modelo interno se muestran asociaciones de referencia, sin afirmar propiedad exclusiva que el código no impone.

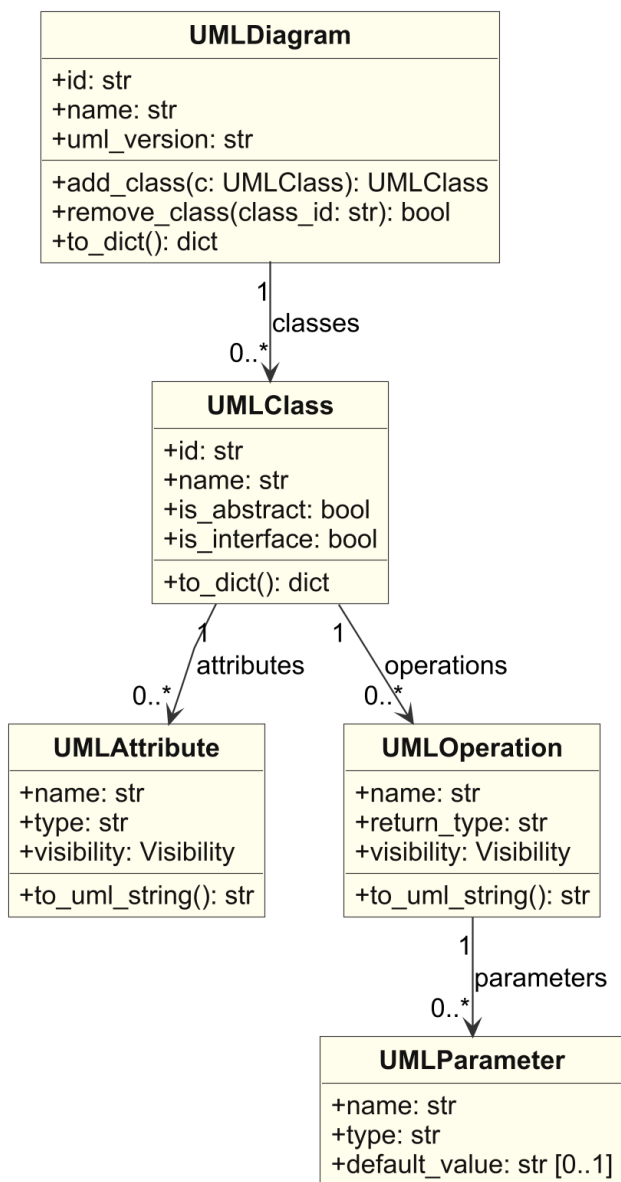
### **5.2 Núcleo del modelo implementado**

UMLDiagram contiene listas de UMLClass y UMLRelationship. UMLClass describe atributos y operaciones; UMLOperation describe parámetros. La separación permite serializar cada elemento y conservar los identificadores usados por edición e intercambio. Los atributos mostrados son una selección legible del código, no una transcripción de todos sus campos.

La figura 3 representa las clases del núcleo y sus asociaciones.

**Figura 3**

*Modelo de clases del núcleo*



Nota. Elaboración propia. Representación del proyecto; los detalles omitidos no implican soporte completo de UML.

### 5.3 Relaciones y extremos

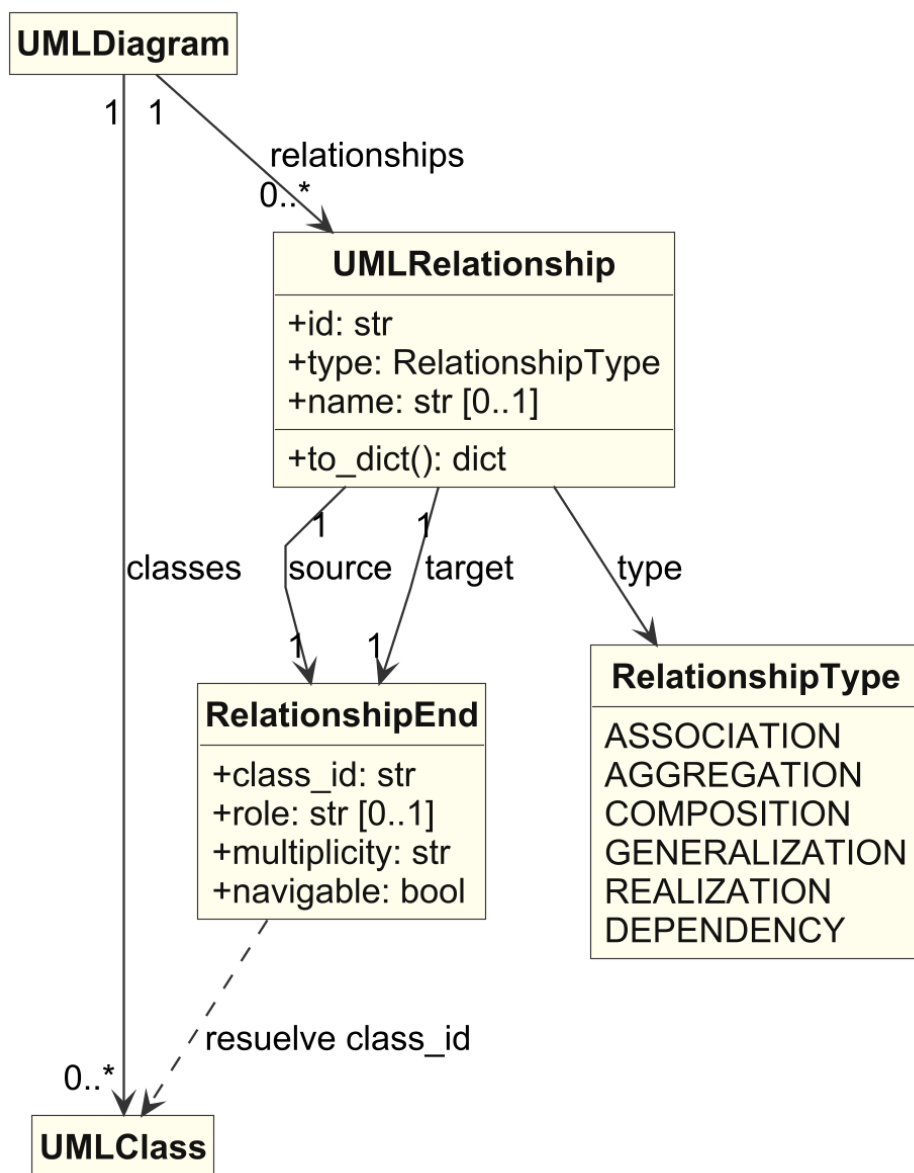
UMLRelationship conserva tipo, origen y destino mediante RelationshipEnd. El extremo referencia una clase por class\_id; no incorpora otra copia de la clase. Esto hace necesaria la

validación de referencias. RelationshipType enumera los seis tipos disponibles. Los campos de presentación, como posición, tamaño y vértices, se omiten de esta vista para concentrarse en la estructura.

La figura 4 representa los extremos y tipos de las relaciones del modelo.

**Figura 4**

*Relaciones y extremos del modelo*



Nota. Elaboración propia. Representación del proyecto; los detalles omitidos no implican soporte completo de UML.

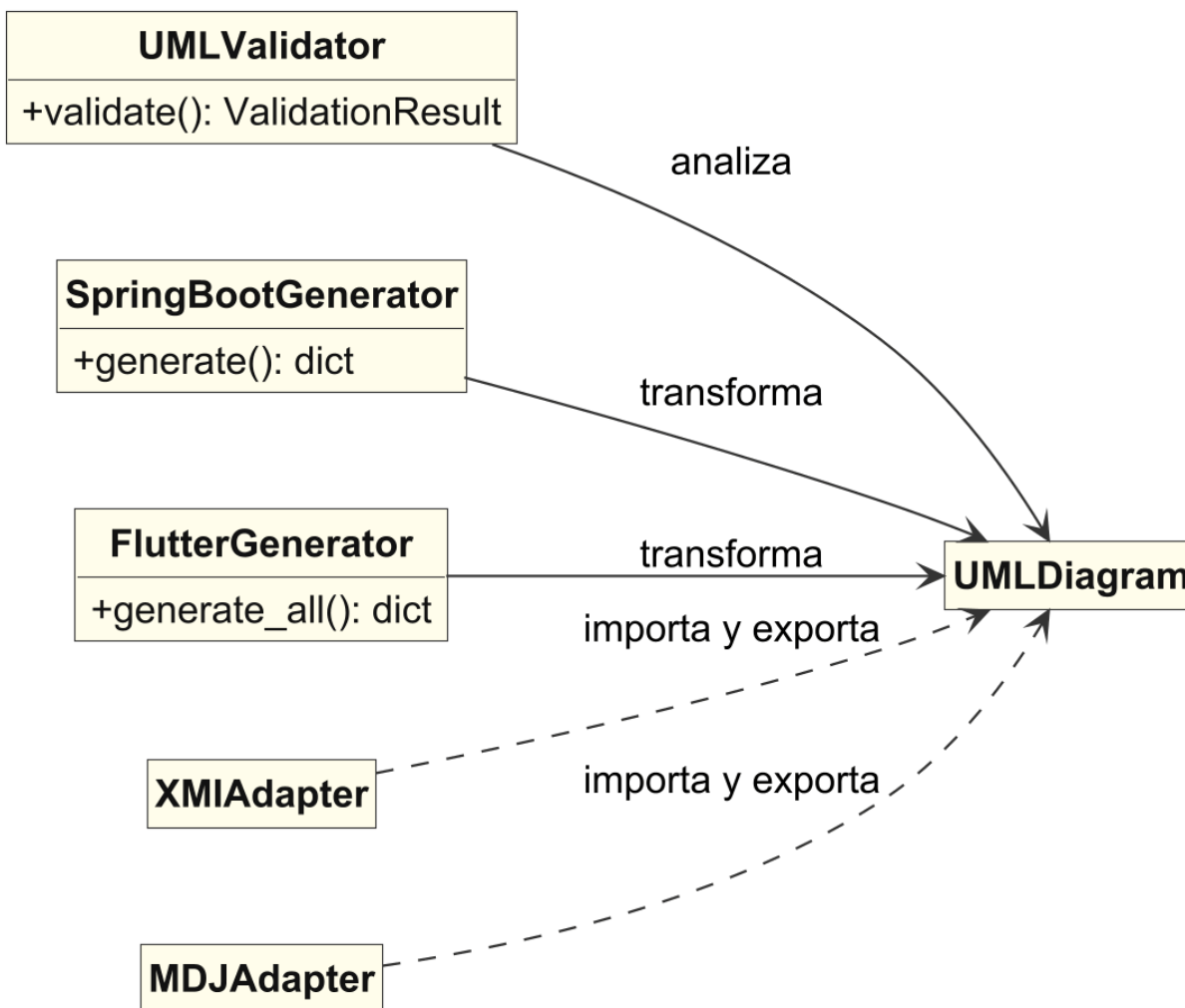
## 5.4 Servicios que consumen el modelo

UMLValidator analiza el diagrama; SpringBootGenerator y FlutterGenerator lo transforman; XMIAAdapter y MDJAdapter convierten archivos. El esquema no inventa una clase base común para esos generadores. Los nombres se contrastaron con backend/app/models y backend/app/services.

La figura 5 representa las dependencias de los servicios respecto del modelo común.

**Figura 5**

*Servicios y dependencias del modelo*



Nota. Elaboración propia. Representación del proyecto; los detalles omitidos no implican soporte completo de UML.

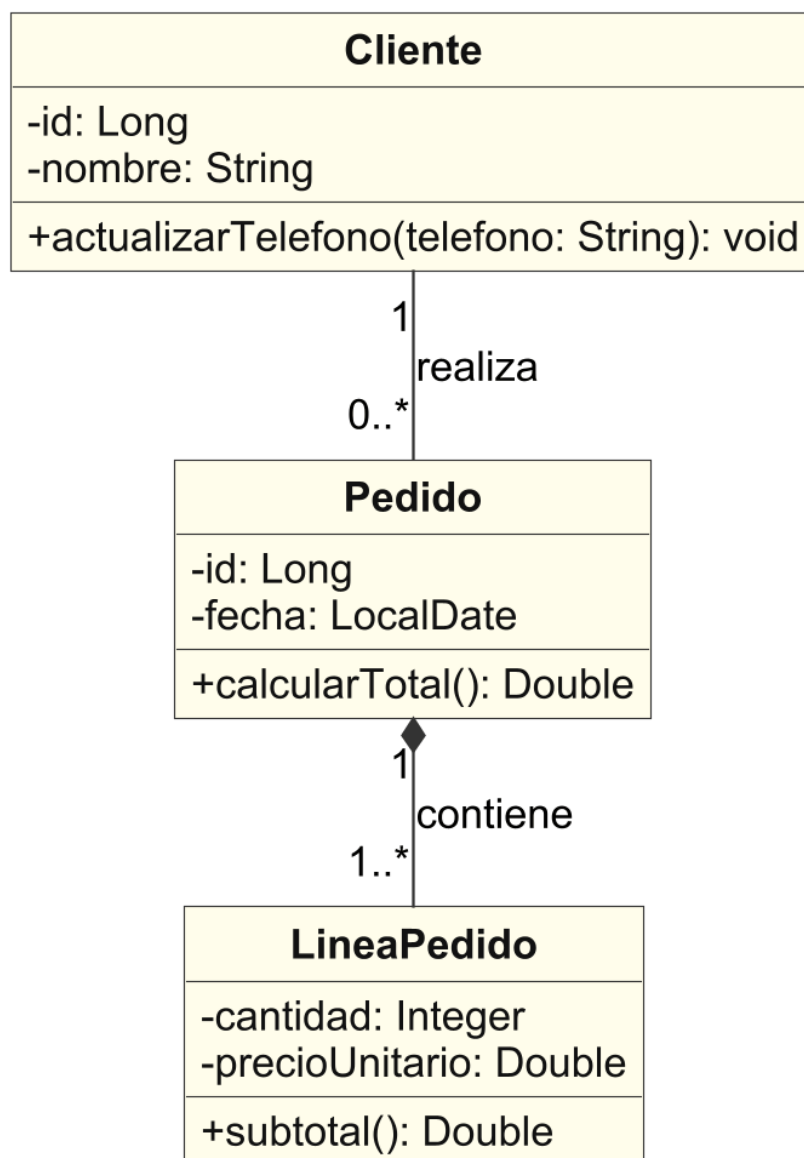
### 5.5 Ejemplo de negocio y reglas

Se propone el ejemplo Cliente, Pedido y LineaPedido para explicar asociaciones, composición y operaciones. Es un ejemplo didáctico, no una captura de una prueba ni el metamodelo del editor. Una regla de negocio posible es que el total del pedido sea la suma de sus líneas; esa regla tendría que implementarse y probarse expresamente. El nombre calcularTotal no basta para generarla con garantía.

La figura 6 representa un ejemplo didáctico de clases de negocio y composición.

**Figura 6**

*Ejemplo didáctico de clases de negocio*



Nota. Elaboración propia. Ejemplo ilustrativo; no corresponde a un resultado de prueba.

## 5.6 Reglas y trazabilidad estructural

Eliminar una clase debe tratar sus relaciones asociadas; importar una referencia inexistente exige validación. Los ciclos de generalización se rechazan en las rutas verificadas.

Las conversiones deben preservar nombres, tipos, visibilidad y extremos según el alcance soportado. Las figuras se vinculan con CU01, CU02, CU07 y CU08.

## 5.7 Transformación del modelo a JPA y PostgreSQL

El mapeo parte de clases y relaciones del modelo conceptual; una tabla relacional no es una clase orientada a objetos. El generador traduce identidad, atributos y asociaciones a entidades y estructuras de persistencia. La implementación de referencia es `backend/app/services/springboot_generator.py`. Las reglas siguientes describen esa implementación y no una obligación universal de UML.

Identidad y atributos. Las clases persistentes se representan como entidades; la identidad se declara mediante `@Id` y, cuando corresponde al identificador generado, `@GeneratedValue` con estrategia `IDENTITY`. Los atributos se convierten en campos y columnas conforme a los tipos admitidos. Una operación del modelo no se convierte en una columna. El tipo ausente no puede considerarse inferido con certeza a partir de una fotografía.

Asociación uno a muchos. Se utilizan `@OneToMany` y `@ManyToOne`; el lado propietario referencia la clave foránea mediante `@JoinColumn` y el inverso utiliza `mappedBy`. Asociación muchos a muchos. Se utiliza `@ManyToMany` y una tabla de unión con las claves de ambos extremos. Asociación uno a uno. El generador utiliza `@OneToOne` con lado propietario e inverso; debe comprobarse la restricción de unicidad efectiva en el esquema obtenido.

Composición y agregación. La implementación actual traduce la composición como colección del contenedor con `CascadeType.ALL` y `orphanRemoval`, junto con una referencia obligatoria desde la parte. La agregación utiliza persistencia y actualización en cascada, sin asumir eliminación de la parte. Estas elecciones son convenciones del generador; no cubren automáticamente todas las combinaciones posibles de multiplicidades UML.

Herencia. La estrategia identificada es JOINED, que distribuye la jerarquía entre tablas relacionadas. Es una decisión de persistencia, no el significado de la flecha de generalización. Debe comprobarse con una jerarquía concreta que se mantienen identidad, referencias y consultas de subtipos.

Restricciones. La nulabilidad, unicidad y claves foráneas deben verificarse en PostgreSQL. Una multiplicidad 1..\* no queda garantizada únicamente por una clave foránea desde la parte: exigir al menos un elemento asociado puede necesitar validación adicional. La documentación no equipara todas las multiplicidades gráficas con restricciones ya implementadas.

### **5.8 Criterio de revisión de UML 2.5.1**

La referencia normativa es Object Management Group (2017). La revisión de clases comprueba nombre, atributos, operaciones, visibilidad, multiplicidades y dirección de generalización. El triángulo hueco apunta al clasificador general; el rombo se ubica en el extremo del todo; el rombo lleno representa composición y el hueco agregación compartida. En el núcleo implementado se muestran asociaciones cuando no hay evidencia de propiedad exclusiva.

La revisión del subconjunto empleado no constituye certificación de soporte de toda la especificación. Los diagramas distinguen estructura implementada, vistas lógicas y ejemplo didáctico. El modelador debe revisar las relaciones recuperadas de fotografías y no aceptar como cierta una conexión ambigua.

## 6 Diseño dinámico y despliegue

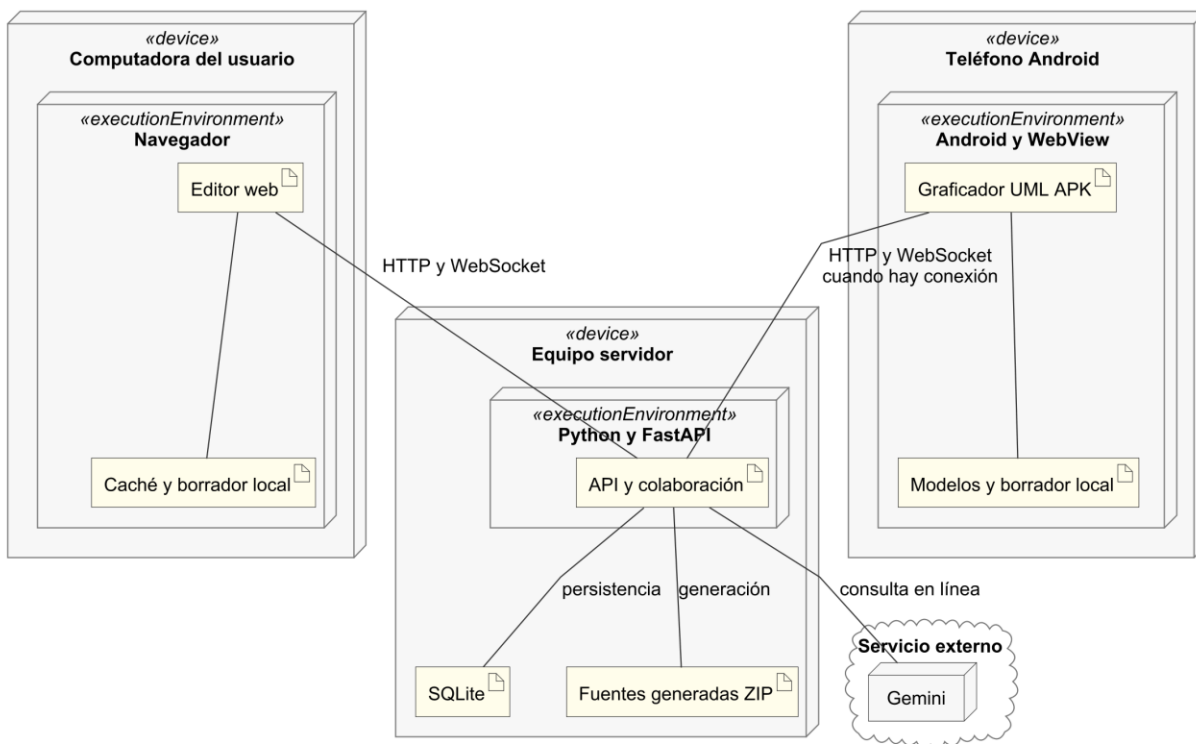
Las secuencias explican el intercambio de mensajes de dos recorridos críticos: importar una foto y reconectar cambios. Las actividades describen el recorrido de generación, incluyendo la validación adicional del proyecto producido. El diagrama de estados resume situaciones de sincronización; no representa clases adicionales del código.

El despliegue distingue computadora, teléfono y servidor. El editor puede conservar operaciones locales sin servidor, pero compartir, consultar Gemini y utilizar la generación del backend requiere conectividad. Las líneas del diagrama representan comunicación lógica; no constituyen una topología de producción ya desplegada.

La figura 7 representa la distribución lógica entre navegador, Android y servidor.

**Figura 7**

*Despliegue lógico web y Android*

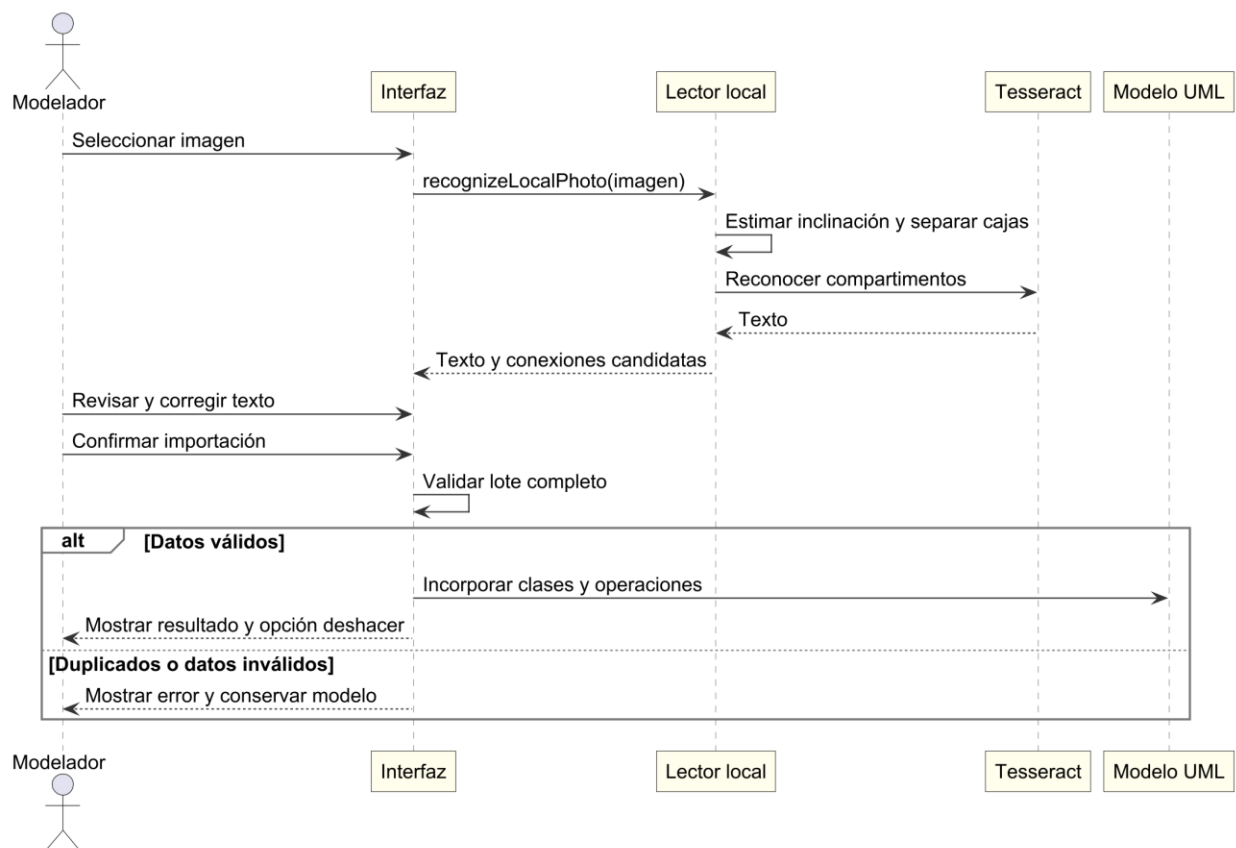


Nota. Elaboración propia. Representación del proyecto; los detalles omitidos no implican soporte completo de UML.

La figura 8 representa la lectura y revisión de una fotografía antes de incorporar el lote.

**Figura 8**

*Secuencia de importación de fotografía*

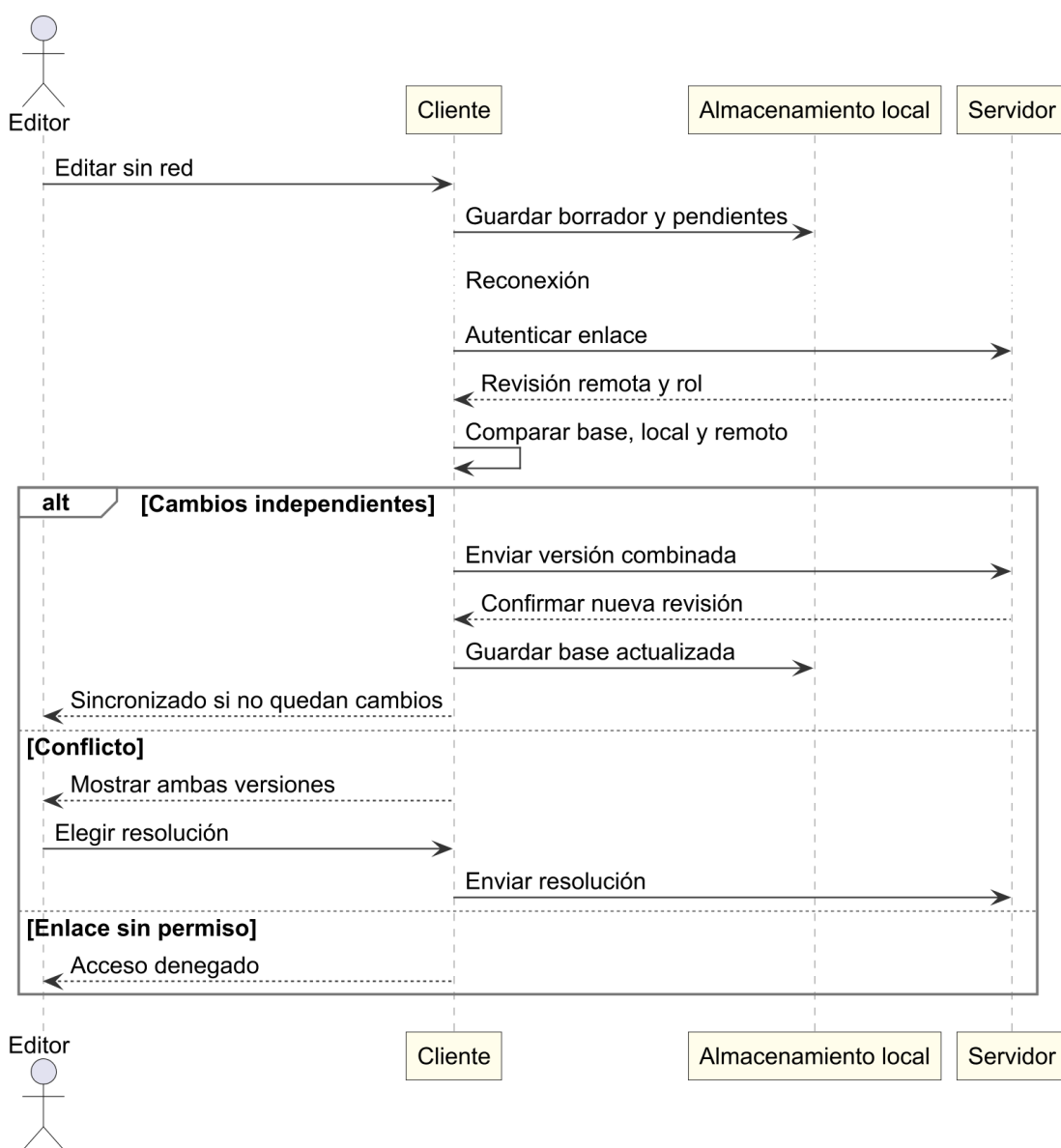


Nota. Elaboración propia. Representación del proyecto; los detalles omitidos no implican soporte completo de UML.

La figura 9 representa la reconexión y los caminos de combinación, conflicto o permiso denegado.

**Figura 9**

*Secuencia de sincronización y conflictos*

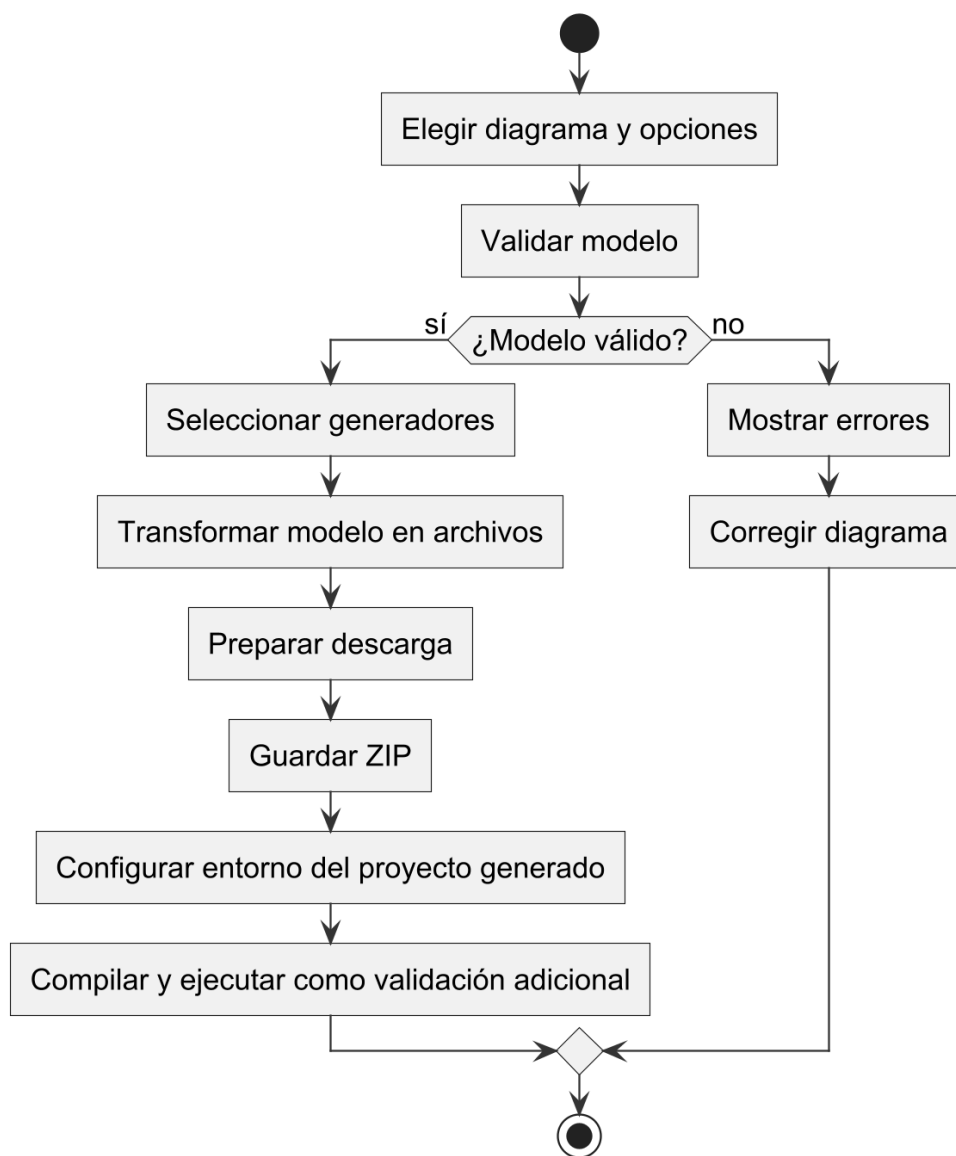


Nota. Elaboración propia. Representación del proyecto; los detalles omitidos no implican soporte completo de UML.

La figura 10 representa el flujo de validación y generación de archivos.

**Figura 10**

*Actividad de generación y validación*

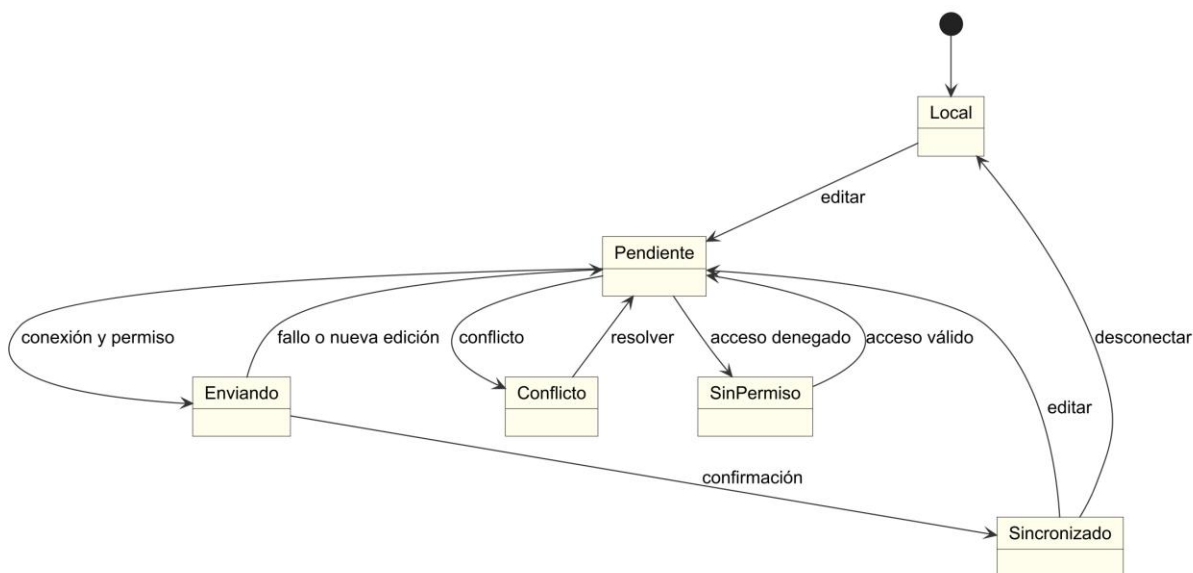


Nota. Elaboración propia. Representación del proyecto; los detalles omitidos no implican soporte completo de UML.

La figura 11 representa los estados conceptuales de sincronización.

**Figura 11**

*Estados conceptuales de sincronización*



Nota. Elaboración propia. Representación del proyecto; los detalles omitidos no implican soporte completo de UML.

## 7 Registro del Proceso Unificado

### 7.1 Criterio del registro

El registro es una reconstrucción basada en el repositorio, los documentos y las pruebas observadas. No se asignan fechas históricas a actividades sin evidencia. Inicio delimita el producto; Elaboración trata arquitectura y riesgos; Construcción produce incrementos; Transición verifica aceptación y prepara la entrega.

### 7.2 Iteraciones reconstruidas

El Proceso Unificado organiza el trabajo en Inicio, Elaboración, Construcción y Transición. Cada iteración integra requisitos, análisis, diseño, implementación y pruebas según el riesgo del incremento. Esta organización toma como referencia la descripción de fases de IBM.

La tabla siguiente reconstruye incrementos a partir de los artefactos disponibles; no inventa fechas de ejecución ni acredita ceremonias que no se hayan registrado.

La tabla 4 presenta iteraciones y evidencia del Proceso Unificado.

**Tabla 4**

*Iteraciones y evidencia del Proceso Unificado*

| <b>Iteración</b> | <b>Fase</b>  | <b>Objetivo e incremento</b>                            | <b>Criterio de salida</b>                                      | <b>Situación documentada</b>                           |
|------------------|--------------|---------------------------------------------------------|----------------------------------------------------------------|--------------------------------------------------------|
| I1               | Inicio       | Delimitar web, móvil y generación; actores y requisitos | Alcance y riesgos identificados                                | Requisitos recuperados de la conversación y documentos |
| E1               | Elaboración  | Modelo UML común y arquitectura cliente/servidor        | Representar y validar clases y relaciones                      | Código del modelo y validador disponible               |
| E2               | Elaboración  | Persistencia, permisos y protocolo colaborativo         | Dos clientes comparten cambios y detectan conflicto            | Pruebas automatizadas existentes                       |
| C1               | Construcción | Editor, formularios e intercambio                       | Modelar y realizar recorridos de importación/exportación       | Implementación y pruebas internas                      |
| C2               | Construcción | Generadores Spring Boot y Flutter                       | Generar proyectos coherentes                                   | Pruebas de generación; ejecución amplia pendiente      |
| C3               | Construcción | APK, voz, fotos y edición local                         | Crear y revisar modelos desde móvil                            | Casos manuales parciales y pruebas automatizadas       |
| C4               | Construcción | Correcciones de OCR, voz y recuperación offline         | No perder borradores ni aplicar datos ambiguos silenciosamente | Cambios recientes probados por componentes             |
| T1               | Transición   | Aceptación, instalación y documentación                 | Verificar criterios finales en entornos objetivo               | Abierta; no declarar concluida                         |

### 7.3 Artefactos por disciplina

La tabla 5 presenta artefactos por disciplina.

**Tabla 5**

*Artefactos por disciplina*

| Disciplina        | Artefactos en este informe o repositorio               |
|-------------------|--------------------------------------------------------|
| Requisitos        | RF01 a RF09, historias, actores y casos de uso         |
| Análisis y diseño | Modelo de clases, componentes, secuencias y despliegue |
| Implementación    | Frontend, backend, adaptadores y contenedor Android    |
| Pruebas           | Suites Python y JavaScript, registros manuales previos |
| Configuración     | Historial Git y cambios locales por integrar           |
| Despliegue        | APK previa y empaquetado posterior pendiente           |

### 7.4 Registro de correcciones como incrementos

Se registraron fallos de transcripción, importación incorrecta de nombres, atributos añadidos a clases equivocadas, indicadores de sincronización engañosos y borrado de cachés de modelos. Las correcciones incorporaron validación previa, revisión del borrador, pruebas de reconexión y conservación de recursos offline. Cada corrección vincula un defecto observado con una prueba; no prueba por sí sola aceptación global.

Las últimas modificaciones incluyen el tratamiento de métodos sin retorno en Flutter, corrección local de inclinación, correcciones de atributos del borrador por voz y ajustes del service worker. No se ha instalado una APK que contenga todas estas modificaciones.

### 7.5 Ciclo de vida y evidencia del proceso

El ciclo de desarrollo conduce de Inicio a Elaboración, Construcción y Transición mediante iteraciones que producen incrementos y permiten revisar riesgos y requisitos. Una

nueva evolución del producto puede iniciar otro ciclo. Las disciplinas atraviesan las fases con distinta intensidad; no se ejecuta primero todo el análisis, después todo el diseño y recién al final toda la programación.

En Elaboración pueden existir modelos antes de completar la implementación, pero terminar esa fase únicamente con modelos y sin validar una arquitectura ejecutable no demuestra que se hayan mitigado los riesgos técnicos. En este proyecto, el modelo común, el protocolo colaborativo y los primeros recorridos de generación sirven como elementos que deben vincularse con la evidencia disponible, sin atribuirles fechas no registradas.

El registro presentado es una reconstrucción apoyada en artefactos y correcciones observadas. Las iteraciones se relacionan con el modelo común, la colaboración, la generación, el cliente Android y las correcciones de reconocimiento. Se distingue la existencia de código de su aceptación mediante pruebas; las fechas no registradas no se reconstruyen como hechos.

## **8 Manual de usuario y tutorial**

### **8.1 Antes de comenzar**

Abra el editor web en la dirección proporcionada por el responsable de instalación o la APK Graficador UML. No confunda esta aplicación con una aplicación de registros generada. Para colaborar, el servidor debe ser accesible desde ambos dispositivos. Para voz, acepte el permiso de micrófono cuando aparezca. Si usa el navegador, las restricciones de seguridad del micrófono dependen del origen y entorno.

Prepare los recursos sin conexión mientras tenga conectividad. El mensaje de preparación no certifica por sí solo la prueba de cierre y reapertura: realice el recorrido del apartado

correspondiente. La interfaz de la APK instalada puede diferir del código más reciente hasta que se actualice.

## **8.2 Tutorial de creación móvil**

1. Abra Crear clase y escriba Persona.
2. Guarde y compruebe que Persona aparece en Clases del Diagrama.
3. Pulse Agregar atributo en esa clase; escriba nombre y elija Texto.
4. Guarde y compruebe el atributo tanto en la tarjeta como en el diagrama.
5. Agregue un método mediante el formulario de operaciones disponible en la versión actual; indique nombre y retorno.
6. Cree otra clase y utilice Conectar clases para elegir extremos y tipo. Revise la dirección antes de guardar.

Resultado esperado: dos clases y una relación visibles. Si un formulario muestra error, corrija los datos; no dé por guardado el cambio únicamente por haber pulsado el botón.

## **8.3 Tutorial web**

1. Seleccione Clase en las herramientas y colóquela en el lienzo.
2. Seleccione la clase y edite su nombre y propiedades.
3. Añada atributos y métodos con nombre y tipo.
4. Cree otra clase y una relación; revise tipo, extremos y multiplicidad.
5. Utilice deshacer si necesita revertir una edición y exporte una copia del modelo.

Resultado esperado: el modelo conserva los elementos introducidos. Antes de reemplazar un diagrama mediante importación, revise la opción elegida y conserve una copia si la necesita.

## 8.4 Tutorial de voz

1. Abra Usar voz y espere a que la interfaz indique escucha.
2. Diga una orden concreta, por ejemplo crear una clase Usuario con atributos nombre y teléfono.
3. Detenga la captura y revise la transcripción.
4. Compruebe la propuesta o confirmación del asistente y el resultado visible.
5. Si el texto no coincide con lo dicho, corrijalo antes de aplicar una propuesta local.

No repita envíos mientras el asistente procesa. Sin conexión, use órdenes admitidas por el intérprete local; no suponga que conserva todas las capacidades de Gemini. Para aislar un fallo, pruebe el mismo texto escrito y compare.

## 8.5 Tutorial de fotografía y corrección

1. Abra Foto u OCR y seleccione una imagen legible con el diagrama completo.
2. Espere el reconocimiento y revise los nombres, atributos y métodos del borrador.
3. Corrija manualmente o dicte: En la clase Usuario cambia el atributo coreo por correo.
4. Compruebe que la modificación ocurrió en la clase adecuada.
5. Revise conexiones candidatas y complete tipo, dirección y multiplicidad cuando sea necesario.
6. Importe después de revisar; si existen nombres duplicados, resuélvalos primero.

El lector intenta corregir inclinación moderada, pero no garantiza recuperación de fotografías con perspectiva, sombras o escritura ambigua. Los tipos ausentes pueden recibir valores predeterminados; no deben interpretarse como datos detectados.

## 8.6 Tutorial de colaboración y reconexión

1. El administrador abre Compartir y elige acceso de edición o lectura.
2. El participante abre el enlace completo en un dispositivo con acceso al servidor.

3. Ambos realizan cambios distintos y comprueban que aparecen en el proyecto compartido.
4. Para evaluar offline, prepare recursos, conserve una copia y quite la conexión de red.
5. Realice una edición, cierre y reabra; compruebe que persiste.
6. Recupere conexión; si aparece conflicto, compare versiones y elija conscientemente.
7. Compruebe el modelo final en ambos dispositivos.

Este recorrido es también un protocolo de aceptación pendiente de ejecución completa en el teléfono. No se presenta como evidencia ya obtenida. Si aparece una advertencia de almacenamiento, no cierre la aplicación antes de conservar una copia.

## 8.7 Exportar y generar

Para guardar el diagrama, abra Exportar y seleccione el formato según el destino. JSON conserva la estructura nativa; XMI y MDJ se destinan al intercambio; los formatos visuales sirven para documentación. Un PNG no sustituye al modelo editable. En Android, seleccione la ubicación de guardado cuando aparezca el selector.

Para producir código, abra el panel de generación, configure las opciones y descargue los archivos. Después revise las instrucciones del proyecto generado y complete su configuración de entorno. Una descarga exitosa no prueba compilación ni funcionamiento completo. Los métodos de dominio pueden contener implementaciones pendientes.

## 8.8 Resolución de problemas

La tabla 6 presenta problemas y acciones de recuperación.

**Tabla 6**

*Problemas y acciones de recuperación*

| Síntoma | Acción del usuario |
|---------|--------------------|
|---------|--------------------|

| Síntoma                   | Acción del usuario                                             |
|---------------------------|----------------------------------------------------------------|
| Voz incorrecta            | Revisar transcripción y probar la orden escrita                |
| Consulta sin red          | Utilizar el intérprete local admitido o recuperar conectividad |
| Foto con texto incompleto | Mejorar iluminación o corregir el borrador antes de importar   |
| Clase duplicada           | Revisar nombres y evitar importar dos veces el mismo lote      |
| Cambios pendientes        | Esperar conexión y comprobar permisos antes de repetir         |
| Conflicto                 | Comparar ambas versiones y conservar la información necesaria  |
| No se pudo guardar        | Mantener abierta la app y exportar una copia si es posible     |
| Modelos no disponibles    | Preparar nuevamente recursos con conexión                      |

## 8.9 Aprendizaje asistido y límites actuales

El manual ofrece recorridos breves para crear una clase, dictar un cambio e importar una fotografía. El asistente facilita introducir órdenes y observar respuestas, pero esto no demuestra por sí solo que exista un tutorial conversacional completo. Para evaluar ese mecanismo, una persona que no conozca la aplicación debe poder identificar la acción siguiente, entender los estados de escucha y revisión, corregir errores y recuperar el trabajo.

El recorrido de aprendizaje recomendado comienza por crear una clase con un atributo; continúa con una corrección guiada y termina guardando y reabriendo el modelo. Deben registrarse dudas y obstáculos observados. No se presenta un tiempo de aprendizaje medido porque no se ha realizado un estudio de usuarios controlado.

## 9 Verificación y aceptación del producto

Las pruebas automáticas verifican contratos y casos definidos; no miden por sí solas facilidad de uso, precisión general de voz o desempeño en cualquier dispositivo. Se distingue entre pruebas de componentes, recorridos internos y aceptación con usuarios y herramientas externas.

La última ejecución completa observada antes de las correcciones posteriores registró 97 pruebas Python y 15 suites JavaScript aprobadas. Después se verificaron 42 pruebas de generación y nuevas pruebas puntuales de foto, corrección y caché. Estos conteos pertenecen a ejecuciones diferentes y no deben sumarse como casos únicos.

### 9.1 Protocolo final de aceptación

La tabla 7 presenta protocolo de aceptación.

**Tabla 7**

#### *Protocolo de aceptación*

| Prueba            | Resultado observable                                         | Estado documental                                       |
|-------------------|--------------------------------------------------------------|---------------------------------------------------------|
| PA01 Modelado     | Crear y conservar clases, atributos, métodos y relaciones    | Evidencia parcial y pruebas automáticas                 |
| PA02 Colaboración | Dos usuarios editan; lector no modifica; conflicto explícito | Servidor y cliente probados; recorrido físico pendiente |
| PA03 Voz          | Captura, transcripción y acción coherentes                   | Casos previos con errores; validación amplia pendiente  |
| PA04 Foto         | Recuperar elementos y revisar relaciones                     | Ejemplo bancario y sintéticos; corpus pendiente         |
| PA05 Offline      | Preparar, cerrar, reabrir y sincronizar sin pérdida          | Componentes probados; Android integral pendiente        |
| PA06 Generación   | Compilar y ejecutar casos representativos                    | Generación probada; validación ampliada pendiente       |
| PA07 Intercambio  | Abrir y reexportar en herramientas destino                   | Recorridos internos probados; prueba externa pendiente  |

## 9.2 Comprobación de backend generado y API

La comprobación del backend generado comprende: revisar entidades y relaciones, configurar la base de datos, compilar, iniciar y probar las operaciones expuestas. En Postman se debe registrar método, ruta real del controlador, cuerpo, respuesta esperada, respuesta obtenida y versión del proyecto. No se incluyen capturas ni colecciones fabricadas para simular una ejecución.

Un caso mínimo crea un recurso válido, lo consulta, lo modifica y lo elimina; después verifica datos inválidos y referencias inexistentes. Para relaciones se comprueba persistencia, restricciones y comportamiento al eliminar. La aprobación requiere la ejecución sobre el proyecto que se entrega, no solo la presencia de anotaciones JPA en el código.

## 9.3 Criterio de cierre

El producto se considera aceptado cuando se ejecutan los recorridos acordados en la versión que se entrega y se registran resultado, entorno, versión y evidencia. La documentación completa no convierte una prueba pendiente en aprobada. El estado de Transición y los límites de la entrega deben ser visibles en la presentación.

## 10 Conclusiones y continuidad

El proyecto integra modelado, reutilización de componentes y transformación de código alrededor de una representación común del diagrama. La arquitectura permite compartir funciones entre web y Android y separar los adaptadores y generadores del ingreso de datos. Las pruebas han permitido detectar fallos concretos y convertirlos en controles de regresión.

El resultado debe presentarse con sus evidencias y límites: las últimas mejoras del código aún requieren empaquetado y validación física, y el reconocimiento general de diagramas y la

interoperabilidad externa necesitan recorridos adicionales. Estas limitaciones constituyen trabajo de aceptación identificado, no resultados que deban ocultarse en la documentación.

## Referencias

Android Developers. (s. f.). *Build web apps in WebView*.

<https://developer.android.com/develop/ui/views/layout/webapps/webview>

Clements, P. C. (1995, 1 de noviembre). *From subroutines to subsystems: Component-based software development*. Software Engineering Institute.

<https://www.sei.cmu.edu/library/from-subroutines-to-subsystems-component-based-software-development/>

Hugging Face. (s. f.). *Use custom models*.

[https://huggingface.co/docs/transformers.js/custom\\_usage](https://huggingface.co/docs/transformers.js/custom_usage)

IBM. (s. f.). *Project planning*. <https://www.ibm.com/docs/en/rational-clearquest/10.0.9?topic=settings-project-planning>

IEEE. (s. f.). *Computer aided software engineering (CASE)*. IEEE Technology Navigator.

<https://technav.ieee.org/topic/computer-aided-software-engineering-case/>

Object Management Group. (2017). *Unified Modeling Language (Version 2.5.1)*.

<https://www.omg.org/spec/UML/2.5.1/>

Software Engineering Institute. (s. f.). *Software architecture*. <https://www.sei.cmu.edu/software-architecture/>

Spring. (s. f.). *Accessing data with JPA*. <https://spring.io/guides/gs/accessing-data-jpa/>

Tesseract OCR. (s. f.). *Introduction*. <https://tesseract-ocr.github.io/tessdoc/Installation.html>

## **Anexo A Inventario de evidencias**

El repositorio constituye la fuente de evidencia de implementación. Los archivos de referencia son backend/app/models/uml\_model.py, backend/app/models/uml\_validator.py, backend/app/services/collaboration.py, los generadores y adaptadores de backend/app/services, frontend/js/local-ai.js, frontend/js/collaboration.js y mobile-android/app/build.gradle.

Las suites relevantes incluyen test\_collaboration.py, test\_generators.py, test\_full\_generation.py, test\_interop\_roundtrip.py, test\_client\_reconnect.cjs, test\_offline\_queue.cjs, test\_photo\_visibility.cjs, test\_photo\_skew.cjs, test\_photo\_voice.cjs y test\_service\_worker\_cache.cjs. El registro de continuidad contiene los resultados observados. No se adjuntan claves, tokens ni enlaces privados de colaboración.

### **Anexo B Guion de demostración**

1. Mostrar el alcance y distinguir editor móvil de aplicaciones generadas.
2. Crear una clase y explicar nombre, atributos, métodos y visibilidad.
3. Conectar dos clases y explicar el sentido de la relación.
4. Mostrar voz y foto con revisión antes de aceptar resultados dudosos.
5. Mostrar colaboración con un editor y un visualizador.
6. Ejecutar el recorrido offline solo si la versión instalada ya fue verificada.
7. Exportar el modelo y mostrar fuentes generadas.
8. Presentar la matriz de aceptación con pendientes identificados.

## **Anexo C Evidencias de prueba y registro de aceptación**

Los registros siguientes consolidan evidencias observadas durante el desarrollo. Las confirmaciones del usuario y las capturas se distinguen de los resultados automatizados. Cuando no consta fecha o identificador de compilación, no se inventa. Para futuras ejecuciones se registrarán identificador, fecha, versión, plataforma, condiciones, pasos, resultado esperado, resultado obtenido y archivo de evidencia.

### **E01 — Importación de clases y atributos desde imagen**

Entrada: diagrama bancario aportado durante la prueba. Plataforma: APK Graficador UML. Resultado observado: el usuario compartió el texto reconocido y capturas del resultado importado con siete clases y veinte atributos. Se identificaron Bank, ATM, Customer, ATMTransactions, Account, CurrentAccount y SavingAccount. Este registro acredita el caso concreto, no una precisión general sobre cualquier pizarra.

### **E02 — Métodos y visibilidad de la imagen**

Entrada: la misma imagen bancaria después de la corrección. Evidencia: texto reconocido y tres capturas de las clases importadas. Resultado observado: diez métodos repartidos entre seis clases y conservación de los signos públicos indicados. Los tipos String y void corresponden a valores predeterminados, porque la imagen no especifica esos tipos. Las relaciones requieren revisión aparte.

### **E03 — Edición manual y persistencia móvil**

El usuario confirmó que agregar correo mediante el formulario actualizó la lista y la representación del diagrama. En otra prueba cerró la aplicación desde recientes y confirmó la

conservación de Usuario y sus atributos. Es evidencia de esos recorridos de guardado; no se extrapola a todos los errores de almacenamiento.

#### **E04 — Voz con interpretación y corrección**

El usuario confirmó creación de Usuario y posterior incorporación de dirección mediante el asistente. También informó errores de transcripción y necesidad de repetir algunas frases. Resultado: funcionalidad demostrada en casos particulares con limitación de reconocimiento; no se registra como precisión perfecta ni como prueba offline aprobada.

#### **E05 — Regresión automatizada**

El registro de trabajo informa una ejecución completa de 97 pruebas Python y 15 suites JavaScript aprobadas, anterior a correcciones posteriores. También informa 42 pruebas de generación aprobadas y comprobaciones puntuales de foto, voz y caché. Son ejecuciones distintas; no se suman como pruebas únicas ni se atribuyen a una nueva ejecución realizada al redactar este documento.

#### **E06 — Límites de la evidencia**

No se adjunta una ejecución integral reciente en Android sin conexión ni una validación nativa completa en StarUML y Enterprise Architect. Estos recorridos están definidos en el protocolo de aceptación del capítulo 9. Documentar su estado permite contrastar el informe con el producto sin inventar resultados ni ocultar fallos.

## **Anexo D Glosario**

CASE: herramientas que apoyan actividades de ingeniería de software. Modelo: representación estructurada de elementos y relaciones. Metamodelo: estructura utilizada para describir los modelos admitidos. OCR: reconocimiento óptico de caracteres. Transcripción: conversión de audio a texto. Conflicto: diferencia que no puede resolverse combinando cambios independientes. APK: paquete de aplicación Android. XMI y MDJ: formatos utilizados por los adaptadores de intercambio del proyecto.

## **Anexo E Preparación de la defensa y trazabilidad**

La evaluación comunicada comprende entrega documental, presentación del producto, contraste entre proceso y software, y demostración de autoría. Estas etapas de evaluación no son las fases del Proceso Unificado. Se prepara el PDF navegable y se conservan las fuentes editables. La indicación de entrega recibida señala el 22 de septiembre a las 08:00 y dos carátulas impresas.

Para demostrar autoría se debe recorrer una solicitud desde la interfaz hasta la validación, actualización del modelo, persistencia y respuesta. Deben explicarse las decisiones tomadas con asistencia de IA y los cambios corregidos después de las pruebas. La generación automática de código no exime de conocer su funcionamiento.

Puntos de defensa: `uml_model.py` define los elementos comunes; `uml_validator.py` comprueba restricciones; `collaboration.py` controla revisiones y permisos; `springboot_generator.py` transforma el modelo en backend; `local-ai.js` participa en los flujos locales. La eliminación de validaciones puede permitir modelos incoherentes; la omisión de persistencia puede perder cambios; alterar permisos puede permitir modificaciones indebidas. Estas consecuencias son análisis del diseño, no experimentos destructivos realizados.

La evidencia debe relacionar requisito, caso de uso, implementación y prueba. CU-01 y CU-02 se vinculan al modelo y validador; CU-03 y CU-04 a permisos, revisiones y reconexión; CU-05 y CU-06 a entrada por voz y foto; CU-07 a adaptadores; CU-08 a generadores; CU-09 al historial. Los resultados pendientes permanecen identificados en el capítulo 9.